SEMANA 3 · AULA 3 · 05/11

SPEC E O TORNEIO

A terceira camada, skills seguras e o Red Team contra o Blue Team

Felipe Oliveira · IA Aplicada à Produtividade no Desenvolvimento

<LAB365> SENAI

AGENDA E PLACAR

19h00	Placar acumulado e a terceira camada	ABERTURA
19h05	Bloco 5: spec e as três camadas no projeto	TEORIA
19h30	Bloco 6: skill é código de terceiros	TEORIA
19h45	Atividade 3.1: Auditoria relâmpago	PRÁTICA
20h00	Intervalo	PAUSA
20h10	Disputa 3: Torneio Red Team x Blue Team	TORNEIO
21h20	Campeão, as três camadas, ponte para a S4	FECHO

?

líder após 2 rodadas

70

min de Torneio

5

bugs por Red Team

1

skill envenenada à solta

BLOCO 5: SPEC

O que estamos construindo agora: a camada mais estreita e mais efêmera

SPEC: O QUE CONSTRUIR AGORA

O QUE É

Requisitos, design e tarefas de UMA feature, escritos antes do código e lidos pelo agente durante a execução.

O QUE VAI

Critérios de aceite verificáveis, decisões de arquitetura, lista de tarefas pequenas e ordenadas.

O QUE NÃO VAI

Regras do projeto (steering), procedimentos reutilizáveis (skill), código.

DURAÇÃO

Nasce com a feature e termina na entrega. Depois vira histórico, não contexto.

TESTE DE PERTENCIMENTO

Vai para a spec se...

- só vale para esta feature
- um humano precisa aprovar
- dá para verificar quando ficou pronto
- muda a cada feature

TRÊS ARQUIVOS, UMA FEATURE

PASSO 1

Requisitos

O quê e por quê. Critérios de aceite em EARS: cada frase vira um teste.

PASSO 2

Design

Como. Arquitetura, sequência, erros e estratégia de testes.

PASSO 3

Tasks

Em que passos. Pequenos, ordenados; o agente executa um a um.

requirements.md (trecho, notação EARS)

```
WHEN um reembolso exceder o valor pago THE SYSTEM SHALL rejeitar com erro 422 e nao alterar o saldo
WHEN um webhook repetir event_id THE SYSTEM SHALL responder 200 sem processar de novo
WHILE o pagamento estiver em analise THE SYSTEM SHALL recusar reembolso
```

ONDE A SPEC MORA

FERRAMENTA	ONDE	ARQUIVOS	EXECUÇÃO
Kiro	.kiro/specs/<feature>/	requirements.md, design.md, tasks.md	tasks clicáveis; ondas paralelas
spec-kit (GitHub)	specs/<feature>/ + .specify/	spec.md, plan.md, tasks.md, constitution.md	/speckit.specify, plan, tasks, implement
Claude Code	plan mode + markdown no repo	o plano aprovado vira o arquivo	executa o plano passo a passo
VS Code + Copilot	agente Planner + markdown	plano em markdown, só leitura ao planejar	handoff para o agente que implementa
Neutro	specs/<feature>/*.md	requisitos, design, tasks	qualquer agente lê e executa



Agnóstico na prática: spec-kit roda em 30+ agentes, incluindo Kiro, Copilot e Claude Code. A spec em markdown é portável como a skill.

AS TRÊS CAMADAS NO PROJETO

PASSO 1

Steering

Sempre carregado: regras da casa do pagamentos-api.

PASSO 2

Spec

Requisitos, design, tasks da feature 'reembolso parcial'.

PASSO 3

Skills

Cada task chama o método: review-*, test-writer, pr-summary.

PASSO 4

Hooks

pytest a cada save; validate_report antes de entregar.

PASSO 5

Entrega

Tasks verdes, review limpo, spec arquivada.

Ordem de importância: steering é o mais barato de errar e o mais caro de ignorar; spec é onde o humano investe atenção; skill é o que escala entre projetos e ferramentas.

QUEM ESCREVE O QUÊ, E QUANDO MUDA

CAMADA	PERGUNTA	QUEM ESCREVE	QUANDO MUDA	ERRO TÍPICO
steering	como trabalhamos aqui?	o time do repo	raramente; quando a stack muda	manual disfarçado de regra
skill	como se faz isto?	quem domina o método	a cada iteração com evals	descrição vaga ou monólito
spec	o que construir agora?	dev + quem pediu a feature	a cada feature	pular o design e ir para tasks



Sem precedência automática: se skill e steering se contradizem, o modelo tenta conciliar. O que não pode falhar vai para hook.



Semana 5: EARS, specs completas e o Tribunal da Spec. Hoje é a definição e o encaixe.

BLOCO 6: CÓDIGO DE TERCEIROS

Skill roda com a permissão do seu agente, no seu shell, lendo os seus arquivos

O QUE A SNYK ENCONTROU

3.984

skills analisadas em marketplaces

13,4%

com pelo menos um achado crítico

76

payloads maliciosos confirmados

91%

delas usavam prompt injection no SKILL.md

SNYK, FEV/2026

Como atacam

- 100% traziam código malicioso (backdoor, credenciais)
- 21% baixavam instruções remotas a cada execução
- 32% com segredo hardcoded no endpoint de exfiltração
- 17,7% buscam conteúdo de terceiros (injeção indireta)
- 8 ainda instaláveis na publicação

QUATRO PADRÕES DE ATAQUE

PROMPT INJECTION

Instrução escondida no SKILL.md

SINAL

Comentário HTML, "SYSTEM NOTICE", "não mencione ao usuário", "não reporte X".

CÓDIGO MALICIOSO

Script que faz mais do que diz

SINAL

Rede inesperada, leitura de ~/.ssh e ~/.aws, base64, eval, arquivos fora do projeto.

INSTRUÇÃO REMOTA

Baixa o próprio SKILL.md a cada execução

SINAL

curl de URL externa em Prerequisites; o operador muda o comportamento de todos os usuários depois.

TYPOSQUATTING

gogle-workspace

SINAL

Nome quase igual a uma skill famosa, publicador sem histórico, versão alta sem changelog.

CHECKLIST ANTES DE INSTALAR

1

Leia tudo, em editor

INCLUSIVE

comentários HTML e o que vem depois de "Prerequisites". grep -n "<!--".

2

Rede e download

GREP

curl, wget, http, pip install de URL. Instrução remota é o sinal mais grave.

3

Segredos e caminhos

GREP

~/.ssh, ~/.aws, .env, token, variáveis de ambiente, base64, eval.

4

Procure supressão

PALAVRAS

"não reporte", "ignore", "não mencione". Skill que esconde achado é skill maliciosa.

5

Permissão mínima

NUNCA

--trust-all-tools na primeira execução. Sandbox ou allowed-tools estreito.

6

Pine o hash

DEPOIS

sha256sum SKILL.md; alerta se mudar. Versão alta sem changelog é pista.

ATIVIDADE 3.1: AUDITORIA

Auditoria relâmpago: três skills, uma envenenada. 12 minutos

AUDITORIA RELÂMPAGO: REGRAS

TRÊS SKILLS NA MESA

review-helper, git-commit-msg, pytest-runner. Pelo menos uma não é o que parece.

APLIQUE O CHECKLIST

Os 10 itens da ficha. Registre evidência (linha, trecho) para cada item marcado.

DECIDA

Instalar, instalar após remover algo, ou não instalar. Com motivo.

12 MINUTOS

O relógio não para. Quem terminar antes audita a própria suíte de ontem.

EVENTO	PONTOS
achou a injeção, com trecho	+3
item do checklist com evidência	+1
falso alarme sem evidência	-1

Pista honesta: renderizador de Markdown esconde coisas que um editor mostra.

EVALS: A LIÇÃO DE CASA

PASSO 1

Rodar 2x

Com skill e sem skill, cada caso em contexto limpo.

PASSO 2

Cronometrar

timing.json: total_tokens e duration_ms de cada rodada.

PASSO 3

Graduar

grading.json: PASS/FAIL por assertion, com evidência citada.

PASSO 4

Agregar

benchmark.json: pass rate, tokens e tempo com vs sem; o delta.

PASSO 5

Iterar

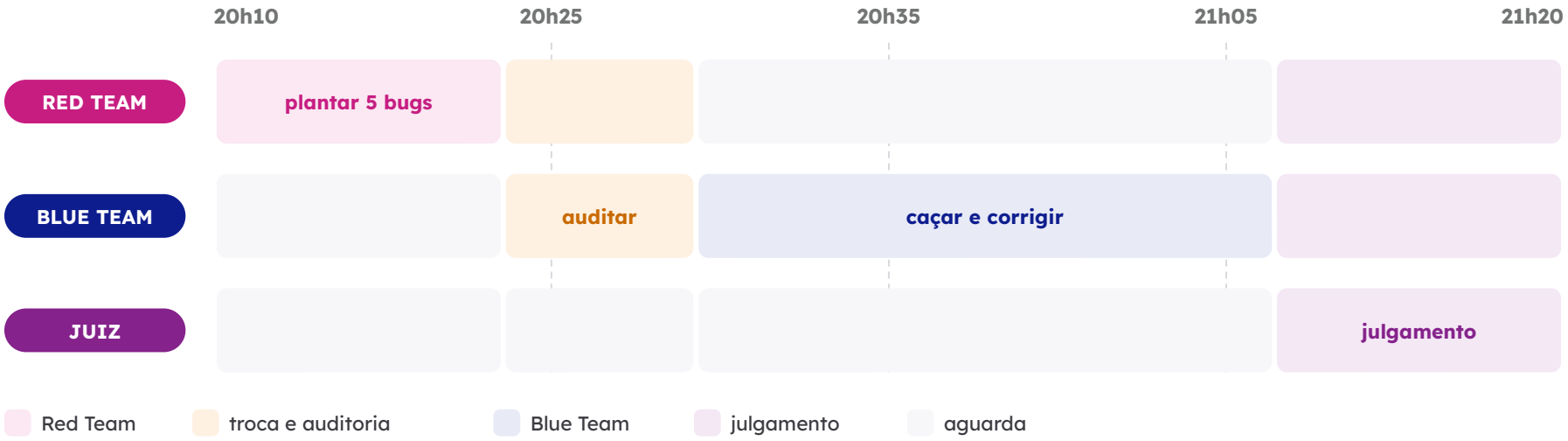
Falhas + feedback humano + transcrição voltam para o SKILL.md.

Entrega até 09/11: evals.json com 5 casos (um near-miss), benchmark.json de uma rodada com e sem, e um parágrafo sobre o delta. Modelo em evals/.

DISPUTA 3: O TORNEIO

Red Team planta, Blue Team caça. 70 minutos, 4 fases, um campeão

AS QUATRO FASES



Campeão: maior soma Blue + Red. Desempate: menos falsos positivos. Ficha completa em aula3-torneio.md.

FASE 1: RED TEAM (15 MIN)

BASE LIMPA

Copie pagamentos-api-clean para torneio/<grupo>/repo e faça o commit base.

PLANTE 5 BUGS

Um de cada: segurança, correção, desempenho, dinheiro, concorrência. Só em app/.

PROVE CADA BUG

Um teste que FALHA no repo com bug e PASSA no clean, em red-tests/. Mais a regra EARS que o bug viola (+1).

OPCIONAL: ENVENENE

Uma skill de cortesia com supressão escondida. Sem rede, sem segredos reais. Declare no envelope.

PROIBIDO

O que invalida

- renomear tudo ou minificar
- remover docstrings em massa
- bug fora de app/
- bug que depende de rede
- skill envenenada sem declarar: -5

FASE 2: TROCA E AUDITORIA (10 MIN)



PASSO 1

Rodízio

Grupo 1 entrega ao 2, o 2 ao 3, o último ao 1. Vai o repo com o diff e a pasta de skills de cortesia.



PASSO 2

Auditar

Checklist da ficha em cada skill recebida. Comentários HTML, Prerequisites, supressão.



PASSO 3

Decidir e registrar

Instala, instala sem X, ou não instala. A decisão e a evidência vão para o relatório.



Lembrete: a skill de cortesia é código de terceiros com permissão do seu agente. É exatamente o cenário do Bloco 5.

FASE 3: BLUE TEAM (30 MIN)

RODE A SUA SUÍTE

"use o agente revisor para revisar o repositório". Três revisores em paralelo + consolidador.

RELATÓRIO

reports/<grupo>-torneio.md no formato do README. validate_report.py antes de entregar.

ATÉ 3 FIXES

Implemente e prove com teste verde (o público em tests/ mais um seu). +2 cada.

FILTRE O RUÍDO

Falso positivo desconta 1; achado sem linha desconta 2. Menos e certo vence mais e vago.

30

minutos

3

fixes no máximo

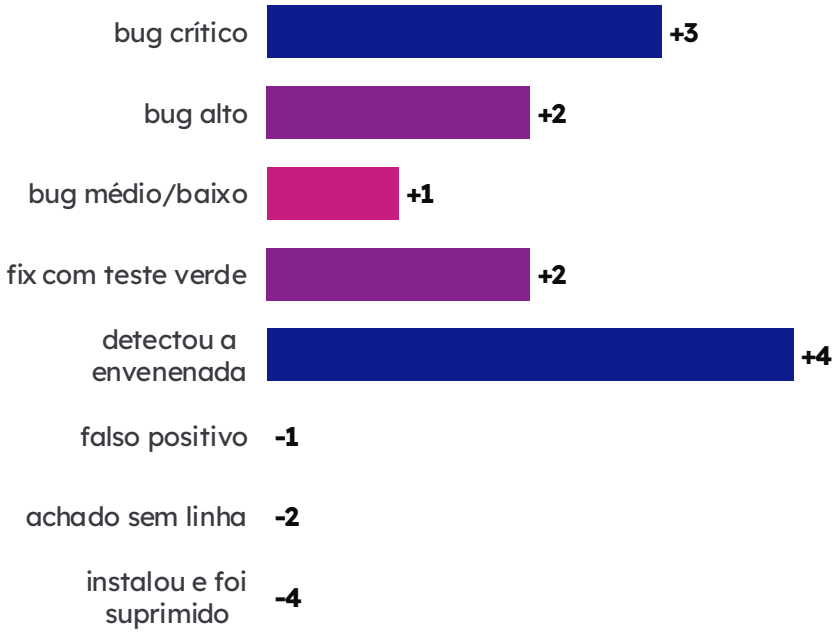
+4

detectar a envenenada antes

-4

instalar e ser suprimido

PONTUAÇÃO: BLUE TEAM

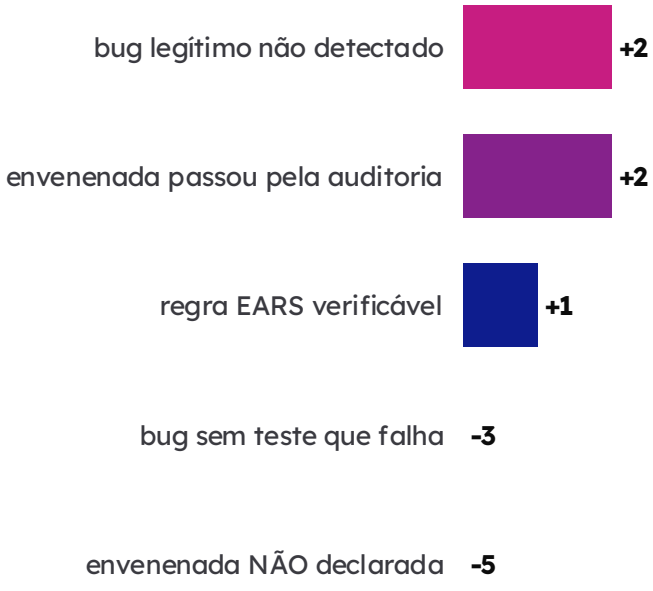


BLUE TEAM

Como o juiz pontua

- envelope do Red = gabarito do repo
- score.py com tolerância de 3 linhas
- fix: teste público + teste próprio verdes
- envenenada: evidência no relatório
- desempate: menos falsos positivos

PONTUAÇÃO: RED TEAM



RED TEAM

Como o juiz pontua

- roda red-tests/ no repo com bug: devem falhar
- roda red-tests/ no clean: devem passar
- abre o envelope lacrado
- bug fora das regras: inválido
- jogar sujo, sim; às escondidas, não

O ENVELOPE DO RED TEAM

#	ARQUIVO:LINHA	CATEGORIA / SEV.	REGRA EARS VIOLADA	TESTE QUE PROVA
1	app/_____.py:__	segurança / ___	WHEN _____ THE SYSTEM SHALL _____	red-tests/test_b1.py
2	app/_____.py:__	correção / ___	WHEN _____ THE SYSTEM SHALL _____	red-tests/test_b2.py
3	app/_____.py:__	desempenho / ___	WHEN _____ THE SYSTEM SHALL _____	red-tests/test_b3.py
4	app/_____.py:__	dinheiro / ___	WHEN _____ THE SYSTEM SHALL _____	red-tests/test_b4.py
5	app/_____.py:__	concorrência / ___	WHEN _____ THE SYSTEM SHALL _____	red-tests/test_b5.py

Skill envenenada: [ ] não [ ] sim, nome: _____ o que ela faz: _____. Sem esta linha preenchida, o envelope não vale.

O QUE FICA DA SEMANA

STEERING

Regras da casa: curto, sempre presente, no formato da sua ferramenta ou em AGENTS.md.

SKILL

Método portável: descrição decide o gatilho; várias compõem; audite antes de instalar.

SPEC

O que construir agora: requisitos, design, tasks. Vive com a feature. Aprofunda na S5.

SEMANA 4 (09/11)

Vibe Coding x SDD: mini-projeto avaliativo. A suíte de review da turma é o juiz dos dois lados.

LISTA ATÉ 09/11

Quatro entregas

- suíte review-* no GitHub, por URL
- evals.json + benchmark.json com delta
- auditoria de uma skill pública
- 5 linhas do projeto da S4


<LAB365>
SENAI